# CSC 4320 / 6320 OPERATING SYSTEMS — EXAM 1 CHEAT SHEET (SIDE 1: CH 1 & CH 2 PROCESSES/IPC/TRACES)

## 1.1 OS FUNDAMENTALS & CATEGORIES

**Two Core Roles:** (1) **Abstraction / Extended (Virtual) Machine**: hides raw HW details; presents files (vs disk sectors), processes (vs raw CPU), virtual memory (vs physical RAM), sockets (vs NIC). (2) **Resource Manager / Control**: allocates competing resources, prevents unauthorized access, coordinates concurrency, handles errors. **Mechanism vs Policy:** **Mechanism** = *"how?"* (e.g., timer, page table, context switch). **Policy** = *"which choice?"* (e.g., which process runs next). **5 OS Responsibilities:** **Process** (create, schedule, sync, terminate), **Memory** (allocate, protect, map, reclaim), **Storage** (name, organize, persist, retrieve), **Devices** (control, buffer, cache, abstract), **Security** (authenticate, authorize, isolate, audit). **Historical Evolution:**

- **Batch**: Goal = *Throughput*; jobs run in groups, no interaction.
- **Multiprogramming**: Goal = *CPU utilization*; keeps several jobs in RAM so CPU switches when one waits for I/O.
- **Time-sharing**: Goal = *Responsiveness*; rapid CPU switching via time slices for interactive users.
- **Real-time (RTOS)**: Goal = *Deadline correctness*. **Hard RT**: missed deadline = system failure (e.g. safety controller; VxWorks, QNX, FreeRTOS [AWS]). **Soft RT**: occasional miss degrades quality (e.g. video frame).

**Multiprogramming Timing Trace (Ch1 S12–15):** *No multiprogramming*: total time = $\sum(\text{Compute}_i + \text{I/O}_i)$ regardless of # of I/O devices (e.g., $P_1(10, 50) + P_2(30, 60) = 60 + 90 = 150$). *With multiprogramming (no time-sharing)*:

- **1 I/O device**: $P_1$ CPU 0 → 10, I/O 10 → 60. $P_2$ CPU 10 → 40, waits for busy I/O device 40 → 60, then $P_2$ I/O 60 → 120 ⇒ ends at **120**.
- **2 I/O devices**: $P_1(20, 40)$, $P_2(10, 80)$. $P_1$ CPU 0 → 20, I/O 20 → 60. $P_2$ CPU 20 → 30, I/O overlaps on 2nd device 30 → 110 ⇒ ends at **110**.

**OS Comparison (S20):** **Windows 11**: NTFS, Windows NT hybrid kernel, PowerShell/WSL, .exe/.msi. **macOS**: APFS, XNU hybrid kernel (Mach-influenced), Homebrew/.dmg. **Linux**: ext4/Btrfs/XFS, monolithic modular kernel, APT/DNF/Pacman. **Mobile** (Android/iOS): aggressive background suspension, ARM, strict app sandboxing. **Cloud-native**: Talos, Fedora CoreOS, AWS Bottlerocket (minimalist, container/microVM).

## 1.2–1.4 ORG, PRIVILEGE & EVENTS

**HW vs SW:** **Physical device** (SSD hardware) ↔ **Device controller** (hardware w/ registers, buffers, DMA) ↔ **Device driver** (OS software translating generic OS requests into controller commands). **Instruction Cycle (von Neumann):** **Fetch** (using PC) → **Decode** (control unit) → **Execute** (ALU/mem) → **Update/Write Back** → **Check Events** (at instruction boundary, check pending enabled interrupt). **Parallel HW:** **Shared-memory** (one address space, loads/stores, locks + cache coherence, multicore). **Distributed-memory** (private RAM per node, messages over interconnect, clusters). **Dual-Mode Operation:** CPU **mode bit** tracks privilege:

- **User mode**: restricted; no privileged instructions, no direct HW/kernel RAM access; crash is isolated.
- **Kernel mode**: full access to HW, page tables, interrupts; crash causes kernel panic / BSOD.

**4 Privileged Instruction Classes:** (1) **I/O control** (start/configure devices), (2) **Memory control** (change page tables/protection), (3) **Interrupt control** (enable/disable interrupts), (4) **Processor control** (halt CPU, change mode). *Attempt in user mode → synchronous hardware trap/exception → kernel terminates or errors.* **Timer:** Kernel programs timer (privileged) → user runs → timer counts down independently → timer interrupt returns control to kernel (prevents runaway infinite loops). **Control Transfers (Events):**

- **Interrupt**: External HW (I/O done, packet arrival, timer); **Asynchronous** (not tied to current instruction, timing varies).
- **Exception / Fault**: Current instruction error/condition (divide by 0, invalid opcode, page fault [recoverable], protection fault); **Synchronous**.
- **Trap**: Intentional software instruction (`syscall`, `int 0x80`, debugger breakpoint); **Synchronous**.

**5 Interrupt-Handling Steps (Order!):** (1) Hardware detects event → (2) Save PC & minimal CPU state → (3) Locate handler via **interrupt vector** → (4) Service the cause (run handler) → (5) Restore state & resume interrupted program.

## 1.5–1.7 SYSCALLS, KERNELS & BOOT

**API vs Wrapper vs System Call:**

- **API**: Application-facing contract (POSIX, Win32, Java); portable across OSs; **may run entirely in user space or invoke 0, 1, or many syscalls**.
- **Library Wrapper**: C library (`glibc`) code that puts syscall # in register (`%rax` on x86-64) + args in registers and executes trap instruction (`syscall`). If kernel returns negative error, wrapper sets `errno` and returns `-1`.
- **System Call**: Kernel-facing protected entry. 6 Categories: **Process** (`fork, exec, exit, wait`), **File** (`open, read, write, close`), **Device** (request, release, control), **Info** (time, PID/UID), **Communication** (pipe, socket, shm), **Protection** (`chmod`, credentials).

**6 Steps of `read(fd, buf, n)`:** (1) App calls wrapper `read()` → (2) Wrapper loads syscall # & args → (3) Trap switches CPU to kernel mode → (4) Kernel validates `fd`, `buf`, permissions → (5) Kernel copies data now OR blocks process for I/O → (6) Wrapper returns > 0 (bytes read), 0 (EOF), or −1 (error, sets `errno`). **strace & File Descriptors (Quiz Forensics):** 0=stdin, 1=stdout, 2=stderr, 3+=first opened file.

- `open("test.txt", O_RDONLY)` ↔ `openat(AT_FDCWD, "test.txt", O_RDONLY) = 3` (or `-1 ENOENT` if missing).
- `perror("open")` ↔ `write(2, "open: No such...", 32) = 32` (writes to fd 2 = stderr).
- `strace` records syscall name, args, and return/errno at the user/kernel boundary, but **cannot directly observe the hardware CPU mode bit or whether a context switch occurred**.

**Mode Switch ≠ Context Switch:** **Mode Switch**: Changes CPU privilege (User ↔ Kernel); **same process/thread continues running**. Every syscall requires a mode switch, NOT necessarily a context switch (only switches process if syscall blocks or quantum expires). **Context Switch**: Changes running process/thread ($P_A$ → $P_B$); saves $P_A$'s CPU state to $\text{PCB}_A$ and restores $\text{PCB}_B$. **Kernel Architectures (S50–58):**

- **Monolithic** (Unix, Linux): All core services in one kernel address space; **direct procedure calls** (fast, no IPC/mode switch between subsystems), large Trusted Computing Base (TCB)—1 buggy driver can crash whole OS.
- **Layered** (THE OS): Strict hierarchical levels; maintainable but rigid/overhead.
- **Microkernel** (MINIX 3, QNX, seL4): Only **IPC, scheduling, basic memory** in kernel; FS, net, drivers run as **user-space servers** communicating via client–server **IPC messages** through microkernel. High fault isolation/reliability/security (small TCB), higher boundary-crossing/IPC overhead.
- **Modular** (Linux LKM): Monolithic core + dynamically loadable kernel modules. **Similarity to hybrid**: core subsystems run in kernel space for speed. **Difference**: Linux modules run in shared kernel space with full privilege (*modular = dynamically extendable, NOT isolated*); hybrid (Windows NT, macOS XNU) uses microkernel-like structure/IPC and can isolate selected services/drivers in user space.

**5 Boot Stages (Exact Order!):** (1) **Firmware (BIOS/UEFI)** in motherboard Flash: runs POST, initializes basic HW, locates boot device (BIOS reads MBR; UEFI reads EFI System Partition, supports GPT & Secure Boot). → (2) **Bootloader** (GRUB, Windows Boot Mgr): selects OS/kernel, **copies kernel (`/boot/vmlinuz`) + `initramfs` into RAM** (CPU needs executable instructions/data in RAM), passes boot params, jumps to kernel entry. → (3) **Kernel Initialization**: configures memory/paging, initializes CPU cores/timers/scheduler, enumerates devices/drivers, mounts root filesystem (`/`). → (4) **Initial User-Space Process (PID 1)**: traditional `init` (sequential scripts, runlevels, daemons) or modern `systemd` (units, dependency graph, parallel startup, targets, logging). *If PID 1 exits, system halts/panics.* → (5) **Login / UI**: launches display manager or terminal login prompt.

## 2.1–2.4 PROCESSES, PCB & IPC

**Program vs Process:** **Program**: passive executable file on disk (no CPU state). **Process**: active executing instance with unique **PID**, virtual address space, CPU state (PC, registers), and OS resources. One program → many processes via multiple launches or `fork()`. **Address Space Layout (Low → High Virtual Addr):** (1) **Text**: executable machine code (read-only, shareable). (2) **Data**: global & static variables (**Data** = initialized; **BSS** = uninitialized/zero-initialized). (3) **Heap**: dynamic memory (`malloc/new`); **grows upward (↑)** toward higher addresses. (4) **Stack**: function call frames, params, return addresses, local variables; **grows downward (↓)** toward lower addresses. Separated by unallocated free virtual space. **5-State & 7-State Lifecycle:** **New** $\xrightarrow{\text{admit}}$ **Ready** $\xrightarrow{\text{dispatch}}$ **Running** $\xrightarrow{\text{exit}}$ **Terminated**. Running $\xrightarrow{\text{preempt}}$ Ready (time slice expires). Running $\xrightarrow{\text{block}}$ **Waiting/Blocked** $\xrightarrow{\text{wake}}$ **Ready** (*never directly to Running!*). **Suspended States (Swapped to Disk):** **Blocked Suspended**: swapped out while waiting for event. When I/O completes → **Ready Suspended** (still on disk!). **Ready Suspended**: logically runnable but on disk; when OS restores/swaps into RAM → **Ready**. **PCB (Process Control Block — Kernel Memory):** 4 Categories: (1) **Identity**: PID, PPID, UID, GID. (2) **CPU Context**: **PC** (checkpoint address of next instruction) & **CPU Registers** (SP, general registers, status flags preserving live computation). (3) **State & Scheduling**: state, priority, queue pointers. (4) **Memory & Accounting/I/O**: page-table pointers, open file descriptors, CPU usage/limits. **PCB & RCB Pointer Trace (Ch2 S23–24):** Initial: $P_8$ running (head of `RL`), $P_8$.next = 7 ($P_7$ ready). $P_7$ holds $R_{15}$ ($P_7$.other_resources = 15 → 25). When $P_8$ requests $R_{15}$: $P_8$.state ← blocked; $P_8$.next ← —; $R_{15}$.waiting_list ← 8; `RL` head ← 7; $P_7$.state ← running. **Priority Queue `create()` / `timeout()` Trace (Ch2 S25–26):** Queues 0, 1, 2 (2=highest). $P_0$ at prio 2 running. `create(n)` puts new PCB at tail of queue $n$ (**does NOT preempt**). `timeout()` moves running process to tail of its prio queue and dispatches head of highest nonempty queue: 1. `create(1)` by $P_0$ ⇒ $P_1$ (prio 1); $P_0$ still runs. 2. `create(2)` by $P_0$ ⇒ $P_2$ (prio 2, behind $P_0$); $P_0$ runs. 3. `timeout()` ⇒ $P_0$ moves behind $P_2$; $P_2$ **runs**. 4. `create(0)` by $P_2$ ⇒ $P_3$ (prio 0); $P_2$ runs. 5. `timeout()` ⇒ $P_2$ moves behind $P_0$; $P_0$ **runs**. 6. `create(2)` by $P_0$ ⇒ $P_4$ (prio 2); $P_0$ has 3 children ($P_1$, $P_2$, $P_4$), $P_2$ has 1 ($P_3$). **`fork()`, `execvp()`, `waitpid()`, Zombie & Orphan:** `pid = fork()`: returns < 0 error, == 0 in **Child** (`getpid()`), > 0 in **Parent** (child's PID). `execvp(prog, args)`: replaces memory image, keeps same PID, never returns on success; on fail call `_exit(127)` ([Note] avoids flushing copied stdio buffers). Parent calls `waitpid(pid, &status, 0)`, checks `WIFEXITED(status)` before `WEXITSTATUS(status)`. **Zombie**: dead child, uncollected exit status. **Orphan**: parent died first; reparented to PID 1. **IPC:** **Shared Memory** (fast, needs sync) vs **Message Passing** (kernel-mediated, isolated). **Unnamed pipe** (`fd[0]` read, `fd[1]` write; related processes; byte stream), **Named pipe/FIFO** (unrelated local via path), **Signal** (async event notice), **Socket** (local/network bidirectional).

# CSC 4320 / 6320 OPERATING SYSTEMS — EXAM 1 CHEAT SHEET (SIDE 2: THREADS, SCHEDULING & CH 3 CONCURRENCY)

## 2.5–2.7 THREADS & CONTEXT SWITCHING

**Thread State:** **Private per thread (in TCB)**: Program Counter (PC), CPU registers, Stack (local vars/call frames), thread state/scheduling info. **Shared across threads in process**: Text (code), Data (globals/statics), Heap, Open File Descriptors. [Note] A thread's stack is NOT hardware-protected from other threads in the same process if a pointer to a local variable is shared! **ULT vs KLT & Mapping Models:**

- **User-Level Threads (ULT / Many-to-One)**: runtime library manages threads without kernel knowledge; fast switch, **0 multicore parallelism** (only **1 core** can run them at once), and **1 blocking syscall blocks the entire process**.
- **Kernel-Level Threads (KLT / One-to-One)**: kernel schedules each thread; runs in parallel on multiple cores; one thread blocking lets others run; higher creation/switch cost.
- **Many-to-Many ($M:N$)**: $M$ user threads over pool of $N$ kernel threads. **Two-Level**: $M:N$ plus ability to permanently bind specific user threads to kernel threads.

**Server Threading Math (Ch2 S53–54):** Arrivals every 25 ms. Cache hit (80%) = 15 ms CPU; miss (20%) = 15 ms CPU + 75 ms disk = 90 ms. Thread creation = 10 ms. **Single-threaded**: blocks on disk $\Rightarrow$ $0.8(15) + 0.2(90) = 12 + 18 =$ **30** ms $>$ 25 ms (unbounded queue!). **Multi-threaded**: overlaps disk wait $\Rightarrow$ CPU time $= 10 + 15 =$ **25** ms $\leq$ 25 ms (keeps up!). **Linux `clone()`, Pthreads & Python GIL:**

- **Linux**: represents both processes and threads as **tasks** (`task_struct`). `clone()` flags decide sharing: `CLONE_VM` (address space), `CLONE_FILES` (fd table), `CLONE_THREAD` (thread group).
- **Pthreads**: `pthread_create(&tid, NULL, fn, &arg)` and `pthread_join(tid, NULL)` (returns 0 on success, error code on fail). Compile: `gcc file.c -pthread`.
- **Python GIL**: limits CPU-bound bytecode to 1 thread at a time (use multiprocessing for CPU-bound; threads work well for I/O-bound).

**Context Switch Mechanics & Overhead:** **5 Steps**: (1) $P_A$ stops executing (interrupt/trap enters kernel) $\rightarrow$ (2) Save $P_A$ CPU state to $\text{PCB}_A$ $\rightarrow$ (3) Scheduler selects $P_B$ $\rightarrow$ (4) Restore $P_B$ state from $\text{PCB}_B$ $\rightarrow$ (5) $P_B$ resumes at saved PC. **Who can cause a context switch for running $p$?** (1) $p$ itself (**True**: blocks/exits), (2) the OS (**True**: preemption), (3) another process $q$ (**False!**). **Thread switch ($T_1 \rightarrow T_2$ same process) vs Process switch ($T_1 \rightarrow T_3$)**: $T_1 \rightarrow T_2$ is *usually* cheaper because address space, page tables, and **TLB** (Translation Lookaside Buffer in MMU caching VPN$\rightarrow$PFN translations) stay warm. **Direct cost**: save/restore regs, scheduler/dispatcher. **Indirect cost**: cold cache, TLB flush, pipeline disruption.

## 2.8–2.11 CPU SCHEDULING & METRICS

**Schedulers:** **Long-term** (admits jobs from Job queue $\rightarrow$ Ready; controls degree of multiprogramming; low freq), **Medium-term** (swaps/suspends & resumes; occasional), **Short-term** (selects Ready $\rightarrow$ Running; very high freq), **Dispatcher** (performs actual switch; **dispatch latency** = time to stop one process and start another). **Non-Preemptive Decision Points (Ch2 S92–93):** Non-preemptive scheduler decides **ONLY** when running process (1) **requests unavailable resource / blocks** (**True**) or (2) **terminates** (**True**). It does **NOT** decide when a new process arrives in RL (**False**) or when a blocked process wakes to Ready (**False**). **Core Formulas:** TAT = CT − AT; WT = TAT − BT; RT = First Start − AT. CPU Util $= \frac{\text{Busy Time}}{\text{Elapsed Time}} \times 100\%$; Throughput $= \frac{\text{Completed}}{\text{Elapsed Time}}$. **Worked Slide Examples (Ch2 S100–141):**

- **FCFS** ($P_1(0,8), P_2(1,4), P_3(2,2), P_4(3,1)$): 0–8 $P_1$ | 8–12 $P_2$ | 12–14 $P_3$ | 14–15 $P_4$. WT $= 0, 7, 10, 11$ (Avg 7); TAT $= 8, 11, 12, 12$ (Avg 10.75).
- **SJF vs SRTF** ($P_1(0,6), P_2(1,3), P_3(3,2)$): **SJF (Non-preemptive)**: 0–6 $P_1$ | 6–8 $P_3$ | 8–11 $P_2$. **SRTF (Preemptive)**: 0–1 $P_1$(rem 5) | 1–4 $P_2$(at $t=3$, $P_2$ rem 1 $<$ $P_3$ rem 2) | 4–6 $P_3$ | 6–11 $P_1$.
- **Preemptive Priority** ($P_1(0,8,\text{p=2})$, $P_2(1,4,\text{p=4})$, $P_3(2,2,\text{p=6})$, $P_4(3,1,\text{p=3})$, higher # = higher prio): 0–1 $P_1$ | 1–2 $P_2$ | 2–4 $P_3$ | 4–7 $P_2$ | 7–8 $P_4$ | 8–15 $P_1$.
- **Round Robin** ($Q=4$, $P_1(0,10)$, $P_2(1,5)$, $P_3(2,8)$): 0–4 $P_1$ | 4–8 $P_2$ | 8–12 $P_3$ | 12–16 $P_1$ | 16–17 $P_2$ | 17–21 $P_3$ | 21–23 $P_1$. CT $= 23, 17, 21$; TAT $= 23, 16, 19$ (Avg 19.33); WT $= 13, 11, 11$ (Avg 11.67). **Rule**: Enqueue arrivals at $t$ *before* re-queuing preempted process!
- **Quantum Overhead Fraction (S128–129)**: Burst $T$, quantum $Q$, switch $S$: If $T < Q \Rightarrow \frac{S}{T+S}$. If $T \gg Q \Rightarrow \frac{S}{Q+S}$.
- **4-Level MLFQ (S139–141)**: $Q_0$(RR 1ms) $>$ $Q_1$(RR 2ms) $>$ $Q_2$(RR 4ms) $>$ $Q_3$(FCFS). $P_1(0,8)$, $P_2(0,3)$, $P_3(1,5)$: $Q_0$: 0–1 $P_1(\rightarrow Q_1)$ | 1–2 $P_2(\rightarrow Q_1)$ | 2–3 $P_3(\rightarrow Q_1)$. $Q_1$: 3–5 $P_1(\rightarrow Q_2)$ | 5–7 $P_2$(Finishes at 7 in $Q_1$!) | 7–9 $P_3(\rightarrow Q_2)$. $Q_2$: 9–13 $P_1(\rightarrow Q_3)$ | 13–15 $P_3$(Finishes at 15 in $Q_2$!). $Q_3$: 15–16 $P_1$(Finishes at 16).

**Multiprocessor, Real-Time & Linux:** **Global vs Per-CPU queue**; **Load balancing** (Push/Pull) vs **Processor Affinity** (Soft/Hard; warm cache & **NUMA** local memory). **RMS** (fixed prio: shorter period $\rightarrow$ higher prio) vs **EDF** (dynamic prio: earliest deadline first, up to 100% util). **Linux: CFS** (smallest `vruntime`) $\rightarrow$ **EEVDF** (kernel 6.6+: eligible tasks w/ nonnegative lag $\rightarrow$ earliest virtual deadline).

## 3.1–3.8 CONCURRENCY & SYNCHRONIZATION

**3.1 Foundations (Exam 1 Oct 7/8 Cutoff at S21!):**

- **Concurrent vs Parallel**: **Concurrent** = overlapping progress over time (interleaving on **1 core** OR multiple cores). **Parallel** = simultaneous execution at the exact same instant (requires $\geq 2$ cores).
- **1-Core Overlap Math (S16–17)**: Task A = 10 ms disk I/O; Task B = 6 ms CPU. Sequential = 10 + 6 = 16 ms. Ideal 2-thread overlap = $\max(10, 6) = 10$ ms (saves 6 ms). With 2 ms coordination overhead = 10 + 2 = 12 ms (net savings = 4 ms).
- **Shared Counter Lost Update (S10–11, S18–20)**: Both threads run `r = counter; r = r + 1; counter = r;`. Here `counter` is shared (global/heap), while `rA, rB` are private thread registers. Interleaving ($A$ reads 10 $\rightarrow$ $B$ reads 10 $\rightarrow$ $A$ writes 11 $\rightarrow$ $B$ writes 11) produces **11 instead of 12** (**lost update**). Can happen on **1 core** via context switch or on multiple cores! With a mutex, either A or B may finish first, but final value is **guaranteed to be 12**.
- **Key Challenges**: Race conditions, Deadlock (circular wait), Starvation (indefinite postponement), **Lost wakeup** (missing signal between checking condition and sleeping), Synchronization overhead, Nondeterministic timing bugs.

**3.2–3.4 Critical Sections, Peterson's & Hardware:**

- **Race Condition vs Data Race**: **Race condition** = correctness depends on uncontrolled event ordering (e.g., check-then-act `if (seats>0) seats--`). **Data race** = unsynchronized conflicting memory accesses where $\geq 1$ writes (**undefined behavior** in C/C++).
- **3 CS Requirements**: (1) **Mutual Exclusion** (*Safety*: $\leq 1$ thread in CS), (2) **Progress** (*Liveness*: thread in remainder section cannot block entry), (3) **Bounded Waiting** (*Liveness*: bound on how many times others may overtake a waiter).
- **Thread-Safe vs Reentrant**: **Reentrant** code uses only per-call state (caller-provided buffer, no static scratch buffer, no nonrecursive internal mutex).
- **Software Attempts**: (1) Single flag `locked`: fails **Mutual Exclusion**. (2) Strict alternation (`turn`): satisfies Mutual Exclusion, fails **Progress**. (3) **Peterson's** (`flag[i]=true; turn=j; while(flag[j] && turn==j);Flag`): satisfies all 3 under sequential consistency (max 1 overtake), but fails on modern CPUs/compilers without memory barriers (`volatile` does NOT fix C/C++ races!).
- **Hardware Primitives**: **Interrupt disabling** only works on 1 CPU in kernel mode. Atomic RMW: **Test-and-Set** (`while(test_and_set(&L))`), **Compare-and-Swap** (`while(CAS(&L,0,1)!=0)`), **Atomic Swap**. Simple spinlocks give mutual exclusion (with **acquire/release** ordering) but **no bounded waiting**.

**3.5–3.8 Semaphores, Monitors & Classical Problems:**

- **Mutex vs Semaphore**: **Mutex** has *ownership*. **Semaphore** tracks permits via `wait(S)/P` and `signal(S)/V`. Init: Mutex=1, Pool=$K$, Event sync (`done`)=0. Blocking semaphore uses `park_and_unlock` to prevent lost wakeups.
- **Monitors & Condition Variables**: Always use `while` **loop**: `lock(m); while(!pred) wait(cv, m); ... unlock(m);` (**Mesa semantics**: signaler continues, another thread may consume state before waiter reacquires lock, or spurious wakeup; **Hoare semantics**: immediate handoff). `signal(cv)` with no waiter is **not stored**.
- **Bounded Buffer**: Init `empty=N, full=0, mutex=1`. Producer: `wait(empty); wait(mutex); insert; signal(mutex); signal(full);`. Consumer: `wait(full); wait(mutex); remove; signal(mutex); signal(empty);`. (*`wait(mutex)` before `wait(full)` causes deadlock!*)
- **Readers–Writers / Dining Philosophers / Barber**: RW: first reader (`readers==1`) `wait(room)`, last (`readers==0`) `signal(room)`. Philosophers: fix deadlock via lower-numbered fork first, $\leq 4$ admitted, or central monitor. Sleeping Barber: FIFO queue ($W$ chairs) + per-customer handshakes (`called, seated, done, left`).
- **Practical Concurrency**: Avoid deadlock via **consistent global lock ordering** (`lower_unique_id` first). **False sharing**: independent writable vars on same cache line. **Thread-Sanitizer**: `clang -fsanitize=thread -g -O1`. **Amdahl's Law**: Speedup $\leq \frac{1}{s + \frac{1-s}{p}}$ ($s = 0.2, p = 4 \Rightarrow 2.5\times$).